

CS 330 · Software Engineering · Class 1 · Tuesday, September 22, 2026

Welcome to Software Engineering

Topic 1 — Foundations & Overview (Introduction)

How this course works · What software is · Why it needs engineering · The four layers · The process framework · Your project

Pressman & Maxim, Software Engineering: A Practitioner's Approach, 9th ed. — Chapter 1 and the opening of Chapter 2



Today, in one hour: three parts, three Active Breaks

Part A · 0:00–0:20

Lesson 1 — Who we are, how we work: 1:2:2, Blocks, the IMP, the two exams

 **Think-pair-share (5')**

Part B · 0:20–0:40

Lesson 2 — What is software? · Lesson 3 — Why engineering? Cost of change, failures

 **Visualize-it +  Fun-quiz (5')**

Part C · 0:40–1:00

Lesson 4 — The four layers · Lesson 5 — The process framework · Your project (IMP)

 **Zen-break +  flash-cards (5')**

Then: 10' break · 100' Studio (team, product, Team Charter + Vision Card = Block 1) · 10' wrap-up and exit ticket





1

Lesson 1

Who we are, how we work

Part A · 0:00–0:15 · then a 5' Active Break

What this course is about

Software engineering is the discipline whose aim is the production of software products that are delivered on time, within an agreed budget, and that satisfy the needs of the client.

On time

A working program delivered a year late is a failed engineering project.

Within budget

Three times the price is a failure even when the code is correct.

Client needs

Doing what was asked — not what was easy to build.

- First half: how professionals organise the work — process, process models, process improvement, agile development
- Second half: the phases of a product — requirements, analysis, design, coding, testing, delivery — and the activities that support them
- Throughout: the professional, ethical and social responsibility of the engineer

Five learning outcomes (LO)

LO1

Argue for and against the industry standards for software processes and process improvement

LO2

Apply design patterns and framework activities to a software design

LO3

Characterise the five types of testing: unit, integration, system, user acceptance, performance

LO4

Analyse and present continuous delivery

LO5

Recognise your professional, ethical and social responsibility

Textbook: Pressman & Maxim, Software Engineering: A Practitioner's Approach, 9th ed., 2020 — chapter numbers refer to this edition.



How a week works: the 1:2:2 rhythm

0:00–1:00	Lecture in 3–4 parts	15' at most per part, then a 5' Active Break
1:00–1:10	Break	Ten real minutes
1:10–2:50	Studio (100')	Your team produces this week's Block; the professor circulates
2:50–3:00	Wrap-up	Block box, due date aloud, exit ticket
At home	Home Sprint (120')	Finish the Block, read the chapters, review flash-cards, Numbers practice

1 : 2 : 2 = one part lecture : two parts Studio : two parts Home Sprint — every teaching Tuesday.



The six Active Breaks — you will see these icons every week



Visualize-it

Sketch the idea just taught, then compare with a neighbour



Try-it

A two-minute hands-on task on your own product



Flash-cards

Pairs quiz each other on the terms of the day



Fun-quiz

Three quick questions — vote with your hands



Zen-break

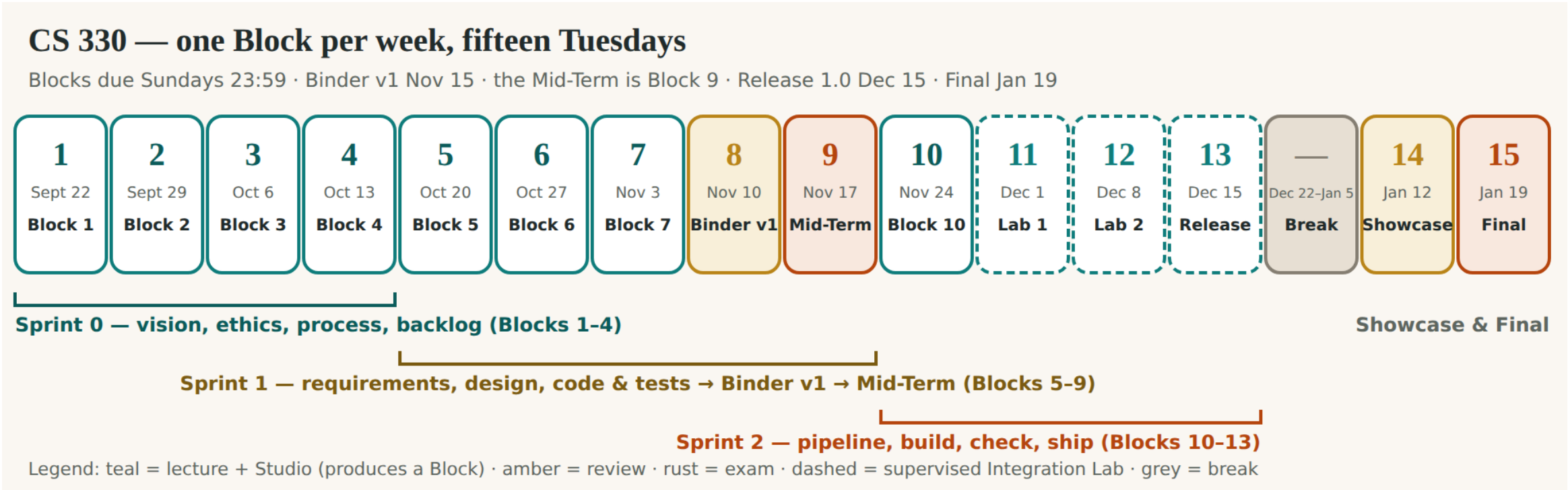
Two minutes of silence and stretching — nothing to do, on purpose



Think-pair-share

One question: a minute alone, two with a partner, three voices for the room

Blocks → Binder → Mid-Term → Release 1.0 → Final



- A Block = the concrete work product of one week: document, diagram, plan, code with tests, release — due the Sunday after its class, 23:59, tagged in the team repository
- Every Block reuses the previous one; nothing you make is thrown away — the Blocks accumulate into your Binder



Read this twice: the Mid-Term is Block 9 of your project

Tuesday, November 17 · 30 % · closed book

- Binder v1 (Block 8) is due Sunday, November 15 — two days before
- The application questions ask you to do, on a product your size, what Blocks 3–7 made you do: choose a process model, size the work, write requirements and a use case, pick a pattern, design tests, compute complexity and coverage
- No notes in the room — your notes are what you internalised by building the Blocks on time
- A late or missing Block is a gap in your preparation; the exam will find it

- **After the Mid-Term — Sprint 2**
- Nov 24: integration and pipeline Block
- Dec 1 · Lab 1 Build it · Dec 8 · Lab 2 Check it · Dec 15 · Lab 3 Ship it → tag Release 1.0
- Dec 21 – Jan 8: winter break
- Jan 12: Showcase and Final review · Jan 19: Final (40 %, LO1–LO5)

How you are graded

30 %

Continuous

13 Blocks $\times$ 2 % = 26 % + 4 % individual engagement

30 %

Mid-Term

Nov 17 · Block 9 · LO1–LO3 · 2 h · closed book

40 %

Final

Jan 19 · LO1–LO5 · 2 h · closed book

- Both exams: individual, closed book, on the Respondus LockDown Browser (online students need a webcam)
- Letter grades (University scale): 93–100 A · 90–92 A- · 87–89 B+ · 83–86 B · 80–82 B- · 77–79 C+ · 73–76 C · 70–72 C- · 60–69 D · 59 and below F
- Engagement (4 %) is individual: fun-quizzes, flash-card sprints, peer reviews, stand-up posts



Rules of the game (1/2)

1 Teams of 3–4

Mixed strengths: someone who codes fast, someone who writes clearly, someone who organises, someone who loves finding bugs

2 The repository is the single source of truth

Documents in /docs, code in the repository, one Git tag per Block (block1, block2, ...)

3 One submission per team

On the LMS: repository link + tag, Sundays 23:59. Late Blocks lose points — and leave a gap in the Binder

4 Everyone contributes visibly

Commits, pull-request reviews, stand-up posts. Engagement is graded individually



Rules of the game (2/2)

5 Academic honesty

Follows the University's policy.

AI tools are allowed only if you declare them in the Block and can explain every line you submit. If you cannot explain it, it is not yours.

6 Course materials are licensed, not given

Study them, keep one personal copy, use them for your Blocks.

Do not re-upload, share on note-sharing sites, feed to a model, or present as your own.

Full rules: [Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy](#) — read it once this week; you will sign its Annex A for this course.

Write on the first page of your notes: 60' lecture · 100' Studio · 120' Home Sprint · one Block every Sunday · 14 Blocks · Binder v1 Nov 15 · Mid-Term Nov 17 · Release 1.0 Dec 15 · Showcase Jan 12 · Final Jan 19



Active Break (5') — Think-pair-share: “Software in my day”

1 minute — alone

List five programs you have used since you woke up (the alarm on your phone counts).

2 minutes — with a partner

Sort your ten programs into categories: system, application, embedded, web or mobile, artificial intelligence, old-and-still-running.

2 minutes — three voices

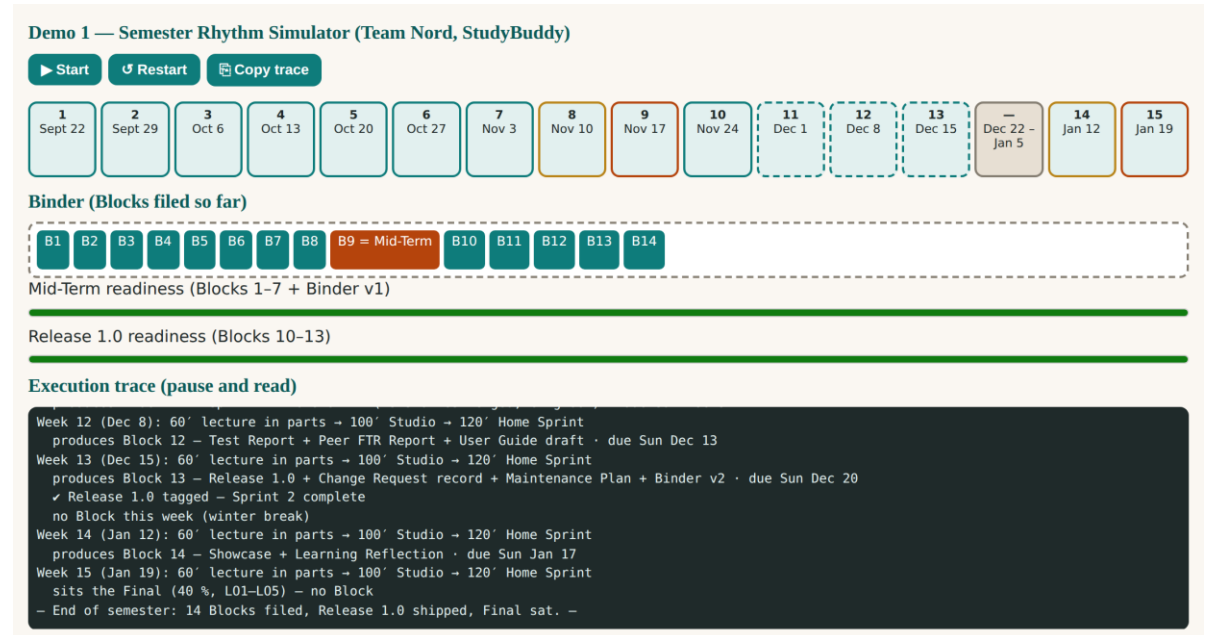
Three people share one surprising category with the room.

Keep your list — Lesson 2 gives the categories their proper names.



Real life — Team Nord builds StudyBuddy, one Block a week

- Four members: Amira (writes clearly), Jules (codes fast), Priya (organises), Théo (finds bugs)
- Sept 22: Team Charter + Product Vision Card → Block 1, due Sunday Sept 27, tag block1
- Each Block is the input of the next: vision → ethics brief → process → backlog → requirements → design → code and tests
- Nov 15: Binder v1 · Nov 17: the Mid-Term, alone, on a product this size
- December: build, check, ship in the three Integration Labs → tag v1.0 on Dec 15
- **Demo 1 — Semester Rhythm Simulator** (in the lecture module): run, pause on any week, read the trace



2

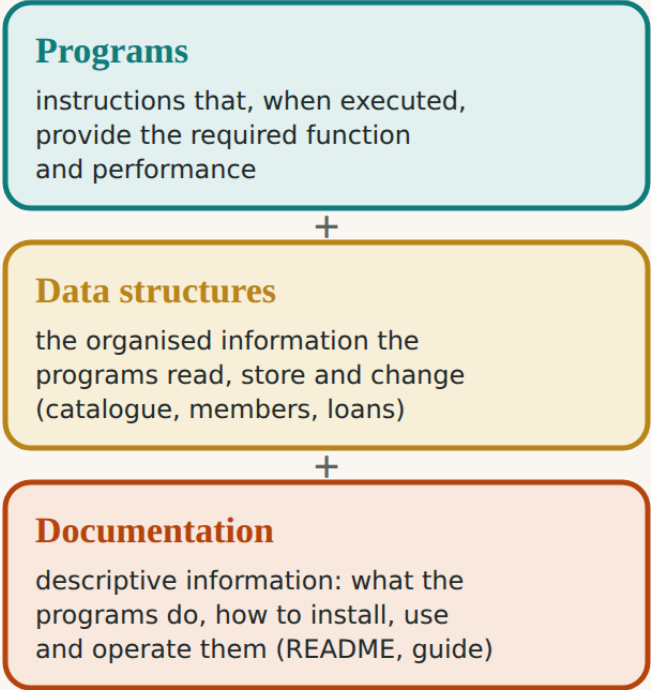
Lesson 2

What is software? Nature and categories

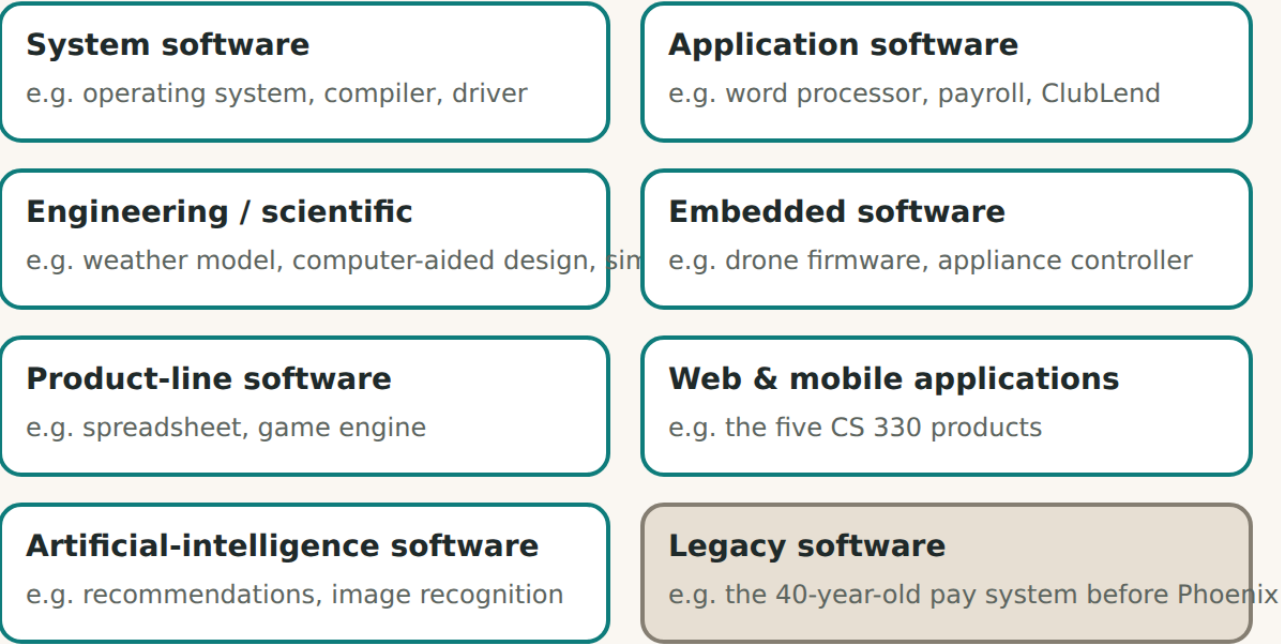
Part B · 0:20–0:27

Software = programs + data + documentation

Software = programs + data + documentation — and the categories of software



Categories (Pressman’s application domains + legacy)



Example: ClubLend (the Robotics Club lending application)

Same size, different category, different quality demands — therefore a different process (Block 3).

Delete any one of the three and the product stops being usable: code without data does nothing useful; data without code is a spreadsheet; neither can be operated without documentation.



Three characteristics that make software different

1 Developed, not manufactured

Once correct, a copy is free and identical.
All cost and risk are in development —
which is why the process matters.

2 Does not wear out — deteriorates

No material degrades with age.
Every change can introduce defects: the
failure rate rises after changes, so we re-test
after every change (regression).

3 Mostly custom-built

Component-based construction (libraries,
frameworks) keeps growing.
The part that makes it StudyBuddy or
ClubLend is still written by you.

The categories of software (1/2)

1 System software

Serves other programs: operating system, compiler, driver

2 Application software

Solves a specific need: word processor, payroll, ClubLend

3 Engineering / scientific

Calculation or simulation at the core: weather model, CAD, structural analysis

4 Embedded software

Lives inside a product and controls it: drone firmware, appliance controller, flight software



The categories of software (2/2)

5 Product-line software

Same core sold to many customers: spreadsheet, game engine, inventory package

6 Web & mobile applications

Browser and phone front ends backed by servers and databases — all five CS 330 products

7 Artificial-intelligence software

Statistical or learned models: recommendations, image recognition, language models

8 Legacy software

Old, business-critical, hard to change (documentation missing, authors gone). Rule one: document before you touch. Example: the 40-year-old pay system before Phoenix

Why the category decides the engineering

A study-partner mobile app

- A defect annoys a student
- Can be released every week
- Light reviews, fast feedback

An insulin-pump controller

- A defect can kill a patient
- Formal reviews, exhaustive testing, certification
- Slow, documented change procedure

Same size, different category, different process. First question of Block 3: what category is our product, and what happens when it fails?



Real life — ClubLend: one loan, three parts of software

- The Robotics Club's old spreadsheet lost its loan history when its owner graduated → legacy software
- Léa searches 'projector': the program queries the data (catalogue) → Projector #2 available
- She asks for 5 days: the program applies documented rules (max 7 days, one projector per member)
- The program writes a loan record (data changes) and quotes the user guide (documentation)
- Five days later: dates compared, reminder sent — a program action, driven by data, following a rule
- **Demo 2 — Anatomy of a Software Product:** each step lights up the part of software at work

Demo 2 — Anatomy of a Software Product (ClubLend: Léa borrows Projector #2)

▶ Start ⚙ Restart 📄 Copy trace

Programs

1. Show the search box
2. Query the catalogue
3. Check availability
4. Apply the lending rule
5. Write the loan record
6. Show the confirmation
7. Compare dates daily
8. Send the overdue reminder

Data

Item	Status
Projector #1	on loan (Omar)
Projector #2	on loan (Léa, due in 5 days)
Drone	available

- Loans: 2 records

Documentation

- README: how to install ClubLend
- Policy: a loan lasts at most 7 days
- Policy: one projector per member
- User guide: "Returning an item"

Categories:

Browser (application)
Server OS (system)
ClubLend (web app)
Old spreadsheet (legacy)
Drone firmware (embedded)

Execution trace (pause and read)

```
6. The program writes a loan record into the DATA and changes the status of Projector #2.
7. The confirmation screen quotes the return date and links to the user guide page 'Returning an item' (DOCUMENTATION).
8. Five days later the program compares today's date with the loan record (DATA) - the projector is overdue.
9. The program sends the overdue reminder: a PROGRAM action, driven by DATA, following a documented RULE. All three parts were needed.
10. - Categories (Lesson 2): sorting the software around the club -
11. Léa's web browser → application software.
12. The operating system of the club's server → system software (it serves other programs).
13. ClubLend → web application (browser front end, server, database).
14. The old loan spreadsheet nobody can maintain → legacy software.
15. The firmware inside the drone → embedded software. End of demo.
- End: programs + data + documentation = software. -
```



3

Lesson 3

Why engineering? Programming vs. software engineering, the cost of change, why projects fail

Part B · 0:27–0:35 · then a 5' Active Break

Programming and engineering are different activities

Programming

- Writing instructions a computer executes correctly
- A genuine skill — most of you have some already
- What the programmer alone controls: the code

Software engineering adds

- **Process** — who does what, when, and what they hand over
- **Quality** — defined, measured and reviewed, not assumed
- **Accountability** — a named team answers to stakeholders: client, users, operators, the public

A student writing a script alone to rename files is programming; a team of four producing a documented, tested, released ClubLend for a real club is engineering.



The IEEE definition of software engineering

“The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software.” — IEEE Std 610.12-1990 (now ISO/IEC/IEEE 24765)

Systematic

A defined sequence of activities — not improvisation

Disciplined

The sequence is followed even under pressure — especially under pressure

Quantifiable

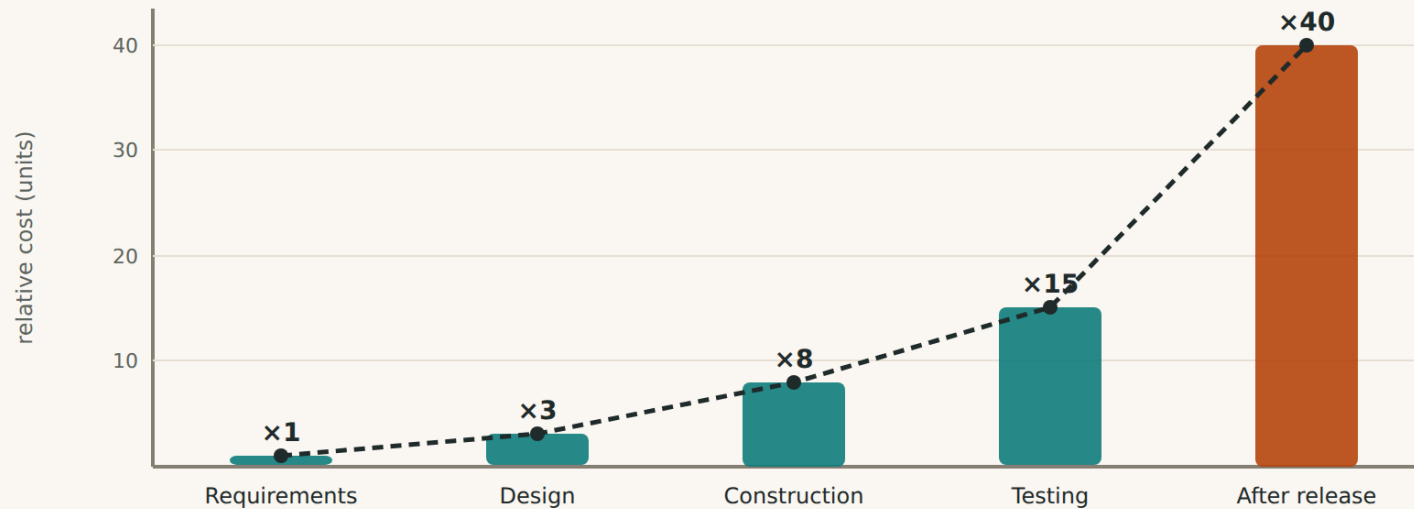
Size, effort, defects, coverage, schedule are measured, so decisions rest on evidence

Three nouns, too: development, operation and maintenance. Engineering does not stop when the code compiles; it covers the years the software is used and changed.

The cost of change grows with the phase

Relative cost of making the same change, by project phase

Illustrative multipliers in line with Boehm's published ranges (up to ~100× for late requirement errors). The shape, not the number, is the le



what must change: sentence → + models → + code & unit tests → + system tests, regression → + redeploy, migrate data, retrain users

Source of the idea: B. W. Boehm, Software Engineering Economics (1981); the multipliers here are teaching values, not measurements.

- Every project changes its mind: a requirement misunderstood, one more field, a new law
- The cost of one change is not constant: it grows with the phase in which it is made
- Boehm's studies: a requirements error fixed after release can cost tens to a hundred times more
- The multiplier depends on the domain; the shape of the curve is the same everywhere

Why: each phase adds dependent work products

1

Requirements

edit one sentence

2

Design

+ redraw the models that depend on it

3

Construction

+ rewrite code and unit tests

4

Testing

+ test plan, system tests, regression run

5

After release

+ redeploy every installation, migrate stored data, retrain users, re-test everything for regressions

Consequences: the cheapest defect is found in a document review (Blocks 5–6 before Block 7); agile methods keep the batches small and the curve flat (Class 4).



Failure 1 — Ariane 5, flight 501

June 4, 1996 — 37 seconds after ignition of the main engine sequence

- Inertial-reference software reused unchanged from Ariane 4
- A 64-bit floating-point value (horizontal velocity) converted to a 16-bit integer — on the faster Ariane 5 trajectory it did not fit
- The exception was not handled; the computer shut down — and its identical backup had already done the same
- Reviews and tests never analysed this unit under Ariane 5 conditions
- Loss of guidance → break-up → self-destruct, with the four Cluster satellites aboard

Who paid for it?

ESA and CNES, and the scientists whose satellites were lost — usually estimated at several hundred million dollars.

Primary source: [ESA press release presenting the Inquiry Board report](#)

Failure 2 — Therac-25

1985–1987 — a computer-controlled radiation-therapy machine

- Software had replaced the hardware safety interlocks of earlier models
- A race condition triggered by fast operator input; error messages nobody understood
- The manufacturer at first insisted the software could not be at fault
- At least six patients massively overdosed in the United States and Canada; several died
- Investigated by Nancy Leveson and Clark Turner — we return to it in Class 2 (responsibility)

Who paid for it?

Patients and their families. The case became a rule that protects people today.

Primary source: [Leveson & Turner, “An Investigation of the Therac-25 Accidents” \(MIT copy\)](#)

Failure 3 — Phoenix, Government of Canada

February and April 2016 — the pay system for about 290,000 federal employees

- To meet schedule and budget: critical pay functions removed, system tests curtailed, the pilot with one department cancelled
- Launched in two waves although warnings had been raised
- Tens of thousands of public servants underpaid, overpaid or not paid at all
- Auditor General (2018): “an incomprehensible failure of project management and project oversight”
- Repair costs have grown into the billions of dollars

Who paid for it?

Employees who could not pay their rent — and every taxpayer.

Primary source: [OAG — 2018 Spring Report 1: Building and Implementing the Phoenix Pay System](#)



What repeats in the three failures

1 Reuse without re-validation

Work reused in a new context without testing it there (Ariane 5)

2 Safety removed, software trusted

Hardware interlocks removed; the software assumed correct (Therac-25)

3 Tests and pilots cut to protect a date

Functions removed, tests curtailed, pilot cancelled (Phoenix)

4 Warnings raised and not acted on

All three

None of these is a failure of programming talent. All are failures of process, quality and accountability — the three things engineering adds. That is why this course exists.





Active Break (5') — Visualize-it + Fun-quiz

Visualize-it (2')

From memory, sketch the cost-of-change curve: phases on the horizontal axis (requirements, design, construction, testing, after release), relative cost on the vertical axis.

Compare with a neighbour: does your curve bend up at the same place?

Fun-quiz (3') — “Is it software engineering?”

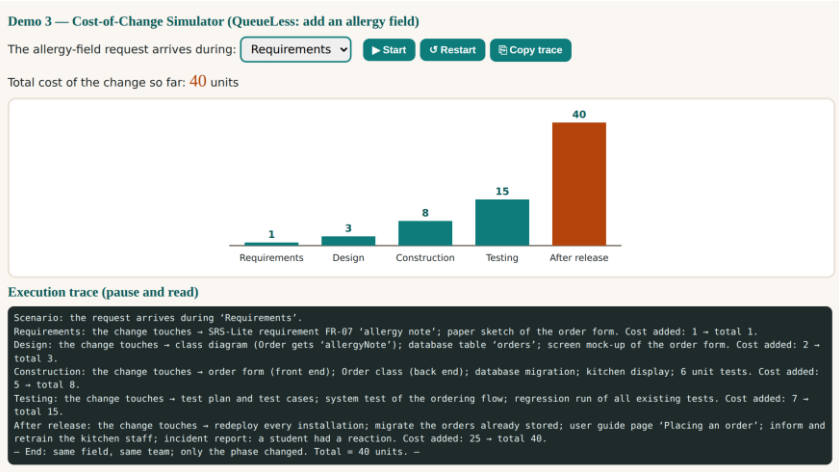
Vote with your hands:

1. A student writes a one-off script alone to rename 200 files.
2. A team of four writes, reviews, tests and releases a room-booking application for a real client with a signed scope.
3. A developer patches a live payroll system on Friday evening without a test and without telling anyone.

Real life — QueueLess: the allergy field that arrived late

Requirements	one line in the SRS + one sketch	≈ 1 unit
Design	+ class diagram, database table, screen mock-up	≈ 3 units
Construction	+ order form, order class, migration, kitchen display, 6 unit tests	≈ 8 units
After Release 1.0	+ full regression, redeploy, migrate stored orders, user guide, inform every user — and a student had a reaction	40+ units

Same field, same team; only the phase changed. Lab 3 (Dec 15): a real change request on your released product, with the cost you measured. Demo 3 lets you choose the phase and watch the bars grow.





4

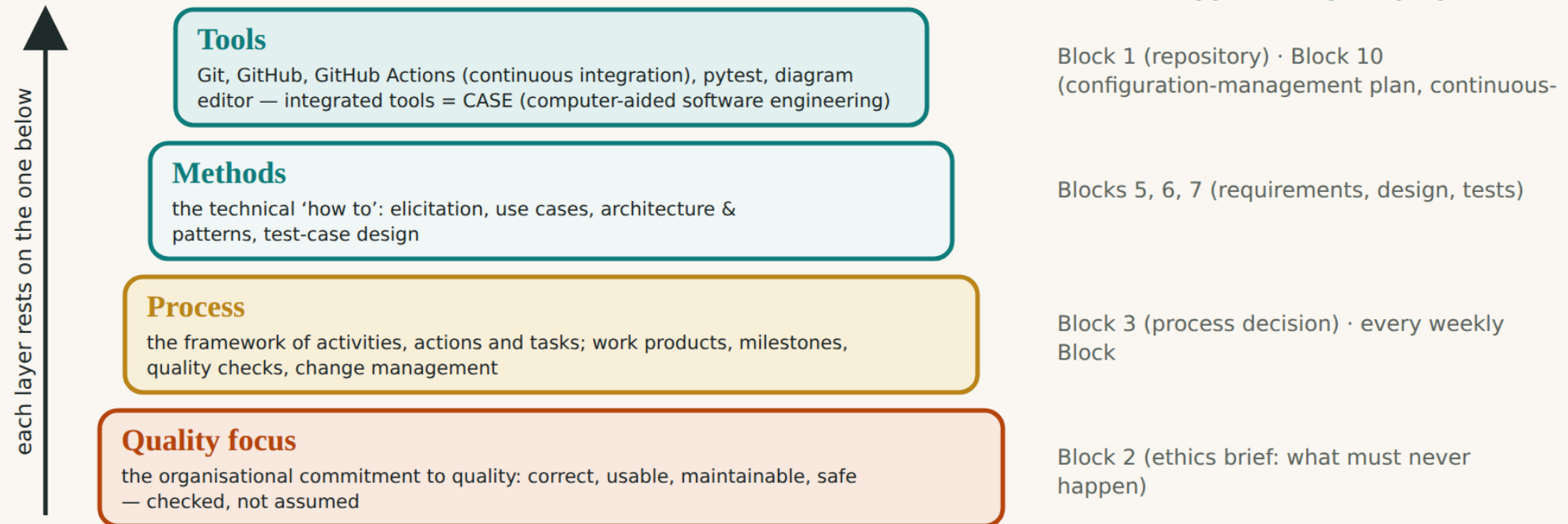
Lesson 4

Software engineering as a layered technology

Part C · 0:40–0:46

Four layers, one direction of dependency

Software engineering as a layered technology — and where you build each layer in CS 330



Common mistake: choosing the tools first. A tool without a method is a purchase, not engineering.

Tools support methods; methods are applied within a process; the process only works when it rests on a quality focus.

Each layer in one line

1 • Quality focus

The organisational commitment to quality — correct, usable, maintainable, safe — checked rather than assumed. Without it the other layers are theatre.

2 • Process

The framework of activities, actions and tasks: work products, milestones, quality checks, change management. The layer management controls.

3 • Methods

The technical 'how to': elicit requirements, model, design, construct, design test cases, support. Notations such as use-case and class diagrams.

4 • Tools

Automated support: Git, GitHub, GitHub Actions (continuous integration), pytest, a diagram editor. Integrated tools = CASE.

The most common mistake of new teams

Built from the top

- “We will use this framework and this pipeline” (tools first)
- Then a method that fits the tool
- Then a process invented around both
- Never asking what quality means for this product
- A tool without a method is a purchase, not engineering

Built from the bottom

- State what quality means for the product (what must never happen)
- Choose the process that protects it
- Choose the methods that produce the evidence
- Only then, the tools that automate those methods
- Every tool now has a reason that traces to a quality statement

Where each layer appears in CS 330

Quality focus → Block 2

Your ethics brief says what must never happen with your product (a report lost, a reporter exposed, a payment charged twice)

Process → Block 3 + every week

Your process model decision record — and the weekly Blocks you live

Methods → Blocks 5, 6, 7

Elicitation and use cases; architectural styles and design patterns; black-box and white-box test design

Tools → Blocks 1 and 10

The repository today; the configuration-management plan and the first green CI pipeline in November



Real life — SafeCampus: choosing the layers before the tools

- Team Ouest's first charter, in the order spoken: GitHub · Python/Flask · pytest · Mondays + review before merge · use cases first · 'a report must never be lost, a reporter never identifiable'
- Sorted: three tools, one process rule, one method — and the quality focus spoken last
- Rewritten from the bottom: quality statement → review checklist that asks 'can this change lose a report?' → use cases and a test table for anonymity → the same tools, now with reasons
- **Demo 4 — Layer Inspector:** classifies each decision and warns when a layer was filled before the one beneath it

Demo 4 — Layer Inspector (SafeCampus: Team Ouest's charter decisions)

▶ Start ⌂ Restart 📄 Copy trace

Tools	GitHub · Python/Flask · pytest
Methods	use cases first
Process	weekly meeting + review before merge
Quality focus	never lose a report · never expose a reporter

Execution trace (pause and read)

Decision 1: "We use GitHub." → layer: TOOLS — automated support for the process (repository, pull requests, boards).
Decision 2: "We code in Python with Flask." → layer: TOOLS — a language and a web framework are tools that support construction.
Decision 3: "We test with pytest." → layer: TOOLS — a unit-test framework automates the testing method.
Decision 4: "We meet on Mondays and review every pull request before merging." → layer: PROCESS — it says who does what, when, and what is checked — a process rule.
Decision 5: "We write use cases before coding." → layer: METHODS — a technical 'how to' for analysing requirements — a method.
Decision 6: "A report must never be lost, and an anonymous reporter must never be identifiable." → layer: QUALITY — a statement of what quality means for this product — the layer everything rests on.
△ 'process' was chosen before 'quality' — the stack was built from the top.
△ 'tools' was chosen before 'methods' — the stack was built from the top.
— Rewrite the charter from the bottom: quality focus → process → methods → tools. Nothing about the tools changes; every tool gains a reason. —



5

Lesson 5

The generic process framework and the umbrella activities

Part C · 0:46–0:52 · IMP launch 0:52–0:55 · then a 5' Active Break

What a process is made of: activity → action → task

1

Framework activity

A broad objective, e.g. 'communication'

2

Software engineering action

A related set of tasks producing a major work product, e.g. 'requirements gathering'

3

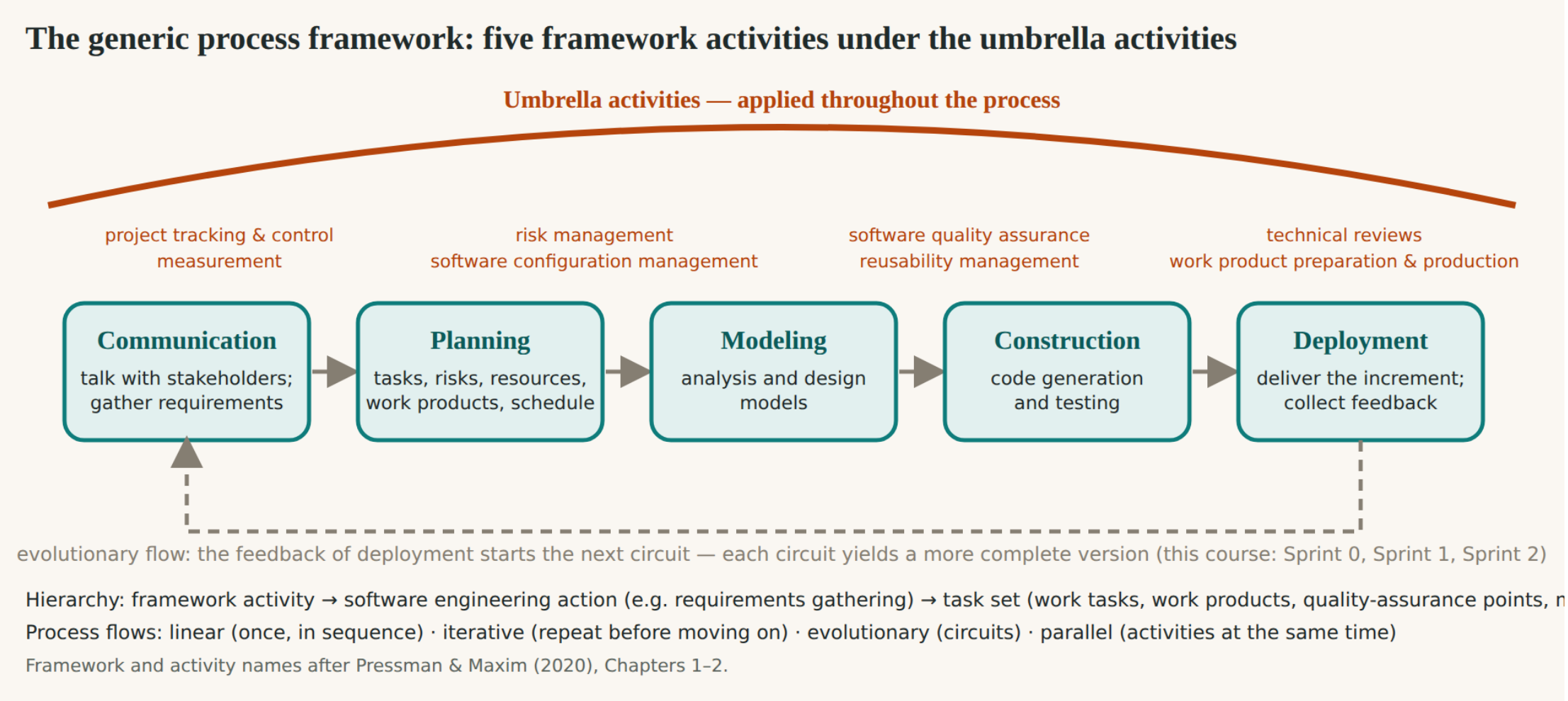
Task set

The concrete work tasks, their work products, the quality-assurance points, the milestones — the level at which one of you does something this afternoon

A software process = the activities, actions and tasks performed when a work product is created. Generic framework = applies to every software project, from a six-week student product to a national pay system.



The five framework activities — memorise them in order



The umbrella activities run across all five, all the time

Project tracking & control

Compare progress with the plan; act on the difference — your stand-up log

Risk management

What may go wrong, and what we prepare — in your sprint plan

Software quality assurance

Ensures the product meets its quality requirements — Block 10, the CI pipeline

Technical reviews

Find and remove errors before they propagate — pull-request review, peer FTR in Lab 2

Measurement

Process, project and product metrics — coverage and complexity in Block 7

Configuration management

Manage the effects of change: versions, baselines — the tag you push every Sunday

Reusability management

Criteria and mechanisms for reuse — the framework you choose in Block 6

Work product preparation

Creating the models, documents, logs and forms — every Block

Process flow: the same five activities, arranged differently

Linear

The five once, in sequence, communication → deployment (the classical waterfall)

Iterative

Repeat one or more activities before moving on

Evolutionary

The five in a circle; each circuit yields a more complete version — this course: Sprint 0, 1, 2

Parallel

Activities at the same time on different parts of the product

Class 3 studies the process models built on these flows. References for the semester: [Pressman & Maxim, 9th ed. \(McGraw Hill\)](#) · [SWEBOK Guide \(IEEE Computer Society\)](#)



Your project (the IMP): the product menu

StudyBuddy

Find study partners by course and book a session; a partner confirms or declines

ClubLend

A club lends equipment and books: catalogue, loans, returns, overdue reminders

QueueLess

Pre-order at the cafeteria and pick up when your number is called; staff see the queue

SafeCampus

Report a hazard or incident (optionally anonymous) and follow its status

HabitHub

Track daily habits and streaks; a weekly report shows what held and what slipped

Your own idea

Pitch it: 6–10 user stories, no payments, no hardware, no sensitive data at scale

Teams of 3–4 · choose in the Studio · scope approved today · small and finished beats big and broken



Three sprints, one product, one semester

Sprint 0

Sept 22 → Oct 18

Vision, ethics brief, process decision, backlog — Blocks 1–4

Sprint 1

Oct 20 → Nov 8

Requirements, design, first code and tests — Blocks 5–7 → Binder v1 → the Mid-Term (Block 9)

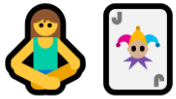
Sprint 2

Nov 24 → Dec 20

Pipeline, build, check, ship — Blocks 10–13, three supervised Integration Labs → Release 1.0

Why on time matters: each Block is the input of the next; a late Block delays a team, and a missing Block is a page missing from the Binder you will need on November 17.





Active Break (5') — Zen-break + first flash-card round

Zen-break (2')

Two minutes of silence.

Feet on the floor, shoulders down, breathe out longer than you breathe in.

Nothing to do — on purpose.

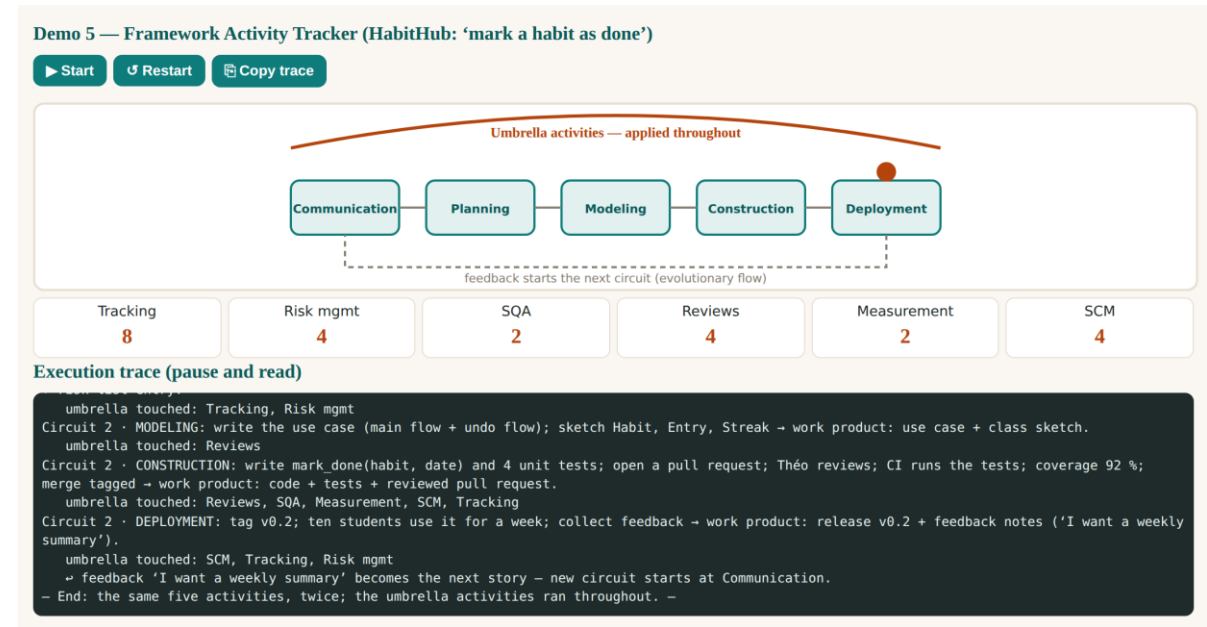
Flash-card round (3') — the eight terms of the day

software · software engineering · quality focus · process
method · tool · framework activity · umbrella activity

In pairs: one term each way, ninety seconds per person.

Real life — HabitHub: one story through the five activities

- Story: “As a student, I want to mark a habit as done today, so that my streak is updated”
- Communication: six interviews → ‘done’ must be reversible (acceptance criterion)
- Planning: 3 points, Sprint 1, risk ‘time zones at midnight’
- Modeling: use case (main flow + undo) and class sketch Habit · Entry · Streak
- Construction: mark_done() + 4 unit tests; pull request; review; CI green; coverage 92 %; merge tagged
- Deployment: tag v0.2, ten users for a week; feedback → the next story — a new circuit
- **Demo 5 — Framework Activity Tracker:** the token moves; the umbrella counters climb



Block 1 — Team Charter + Product Vision Card (summary)

Team Charter

Names and contacts

Four rotating roles: Coordinator, Scribe, Builder, Tester

Meeting cadence · tools (GitHub + one chat channel)

Three working agreements · one conflict rule

Product Vision Card

“For [users] who [need], [product] is a [category] that [key benefit].
Unlike [alternative], our product [difference].”

If it needs two breaths to read aloud, it is too big.

Sixty-second pitches close the Studio

The full Block 1 brief (templates, rubric) is issued separately

Where: /docs/01-charter-vision.md in the team repository · repository link + member list on the LMS · tag block1

Due: Sunday, September 27, 2026, 23:59 (course local time) · **Weight:** 2 % · **Feeds:** everything — your product is the object of every later Block



Home Sprint (120') and the exit ticket

- Create your GitHub account if needed, and the team repository: README = Vision Card; a /docs folder; add all members and the professor as collaborators
- Read Pressman & Maxim, Chapter 1; skim the first pages of Chapter 2
- Watch the videos of Lessons 1–5 and run the five demos once each; read at least one demo's code
- Make your first eight flash-cards; take the five formative quizzes in the lecture module (not graded)
- Finish and submit Block 1

Exit ticket (2:50–3:00)

One sentence on paper:

“What is the difference between programming and software engineering?”

Hand it in on your way out — it shapes the first five minutes of Class 2.



The eight terms of the day — your first flash-cards

Software

programs + data structures +
documentation

Software engineering

a systematic, disciplined,
quantifiable approach to
development, operation and
maintenance

Quality focus

the organisational commitment to
quality on which all other layers rest

Process

the framework of activities, actions
and tasks; work products,
milestones, checks, change

Method

the technical 'how to' for a task

Tool

automated support for process and
methods

Framework activity

communication · planning ·
modeling · construction ·
deployment

Umbrella activity

tracking, risk, SQA, reviews,
measurement, SCM, reuse, work
products — applied throughout

Watch at home — one short video per lesson (each ≤ 15 min)

- Lesson 1 — [Software Engineering — Crash Course Computer Science #16](#) (CrashCourse · 10 min 22 s)
- Lesson 2 — [System Software vs Application Software – What's the Difference?](#) (YouTube · short explainer)
- Lesson 2 — [Operating Systems — Crash Course Computer Science #18 \(system software, in depth\)](#) (CrashCourse · ≈ 13 min 30 s)
- Lesson 3 — [ARIANE 5 First Flight Failure — Flight 501, June 4, 1996 \(launch footage\)](#) (YouTube · short archival clip)
- Lesson 3 — [Software Engineering — Crash Course Computer Science #16 \(why large programs need engineering\)](#) (CrashCourse · 10 min 22 s)
- Lesson 4 — [Software Engineering – Layered Technology \(Quality Focus, Process, Methods, Tools\)](#) (YouTube · short lecture)
- Lesson 5 — [#6 Process Framework — 5 Framework Activities |SE|](#) (YouTube · 8 min 16 s)
- Lesson 5 — [#7 Process Framework — the 8 Umbrella Activities |SE|](#) (YouTube · 7 min 51 s)

Third-party videos — no affiliation or endorsement. Read the address before you click and scan it with [VirusTotal](#) or [Google Safe Browsing](#).



Links and references (official sites — scan before you click)

- [Pressman & Maxim, Software Engineering: A Practitioner's Approach, 9th ed.](#)
- [SWEBOK Guide — IEEE Computer Society](#)
- [Software Engineering as a Layered Technology — IEEE Computer Society](#)
- [ESA — Ariane 501 Inquiry Board report \(press release\)](#)
- [ESA Science — the Inquiry Board report](#)
- [Leveson & Turner — Therac-25 accidents \(MIT copy\)](#)
- [OAG Canada — 2018 Spring Report 1: Phoenix Pay System](#)
- [Git](#)
- [GitHub](#)
- [GitHub Actions](#)
- [pytest](#)
- [Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy](#)



See you in the Studio

Form your team · choose your product · write the Team Charter and the Vision Card · Block 1 due Sunday, September 27, 23:59

Educational material — Cyber.SoHo Educational Hub. © 2026 Cyber.SoHo. All rights reserved. Licensed to enrolled CS 330 students for personal study and for completing the Blocks; not for re-upload, sharing or use as training data. No affiliation with any publisher, platform, vendor or institution named in these slides. Full terms: [CyberSoHo Legal Notice, Terms of Use and Acceptable Use Policy](#)